NY Times API

Introduction

In order to fulfill the requirement of the assignment I have followed the following steps:

Stage 1: Account Registration and API Key Acquisition

- **What I have done:** Setting up my access pass to the NYT database and creating the login at https://developer.nytimes.com/.
- Click + **New App**. Give my app a name as "Book API", enabled the **Books API** product, and click **Save**. Click on my newly created app from the list and copy the generated **API Key** string and Kept this handy; As I will need it for Stage 2.

Stage 2: Secure Credential Management

- **What I have done:** Ensuring my private API key is hidden from plain text so that my code remains secure and reproducible (especially if you push your code to GitHub).
- For that purpose, opened my R console and ran usethis::edit_r_environ() to open mylocal .Renviron file. where I added my key.
- Save the file and restart my R session.
- In my Quarto document, I will later pull this key dynamically via Sys.getenv("NYT_API_KEY") rather than typing the key out explicitly.

Stage 3: Defining the Analytical Question

What have done: Framing my data goals before writing code so my report tells a clear, cohesive story.

- **Selected Endpoint**: The NYT Books API Hardcover Fiction current list ([https://api.nytimes.com/svc/books/v3/lists/current/hardcover-fiction.json]
- **Research Question**: "Which publishers dominate the current New York Times Hardcover Fiction bestseller list, and how do their publication trends break down by ranking and longevity (weeks on the list)?"
- Writing this out clearly in the introductory text section of my Quarto document satisfies the requirement to ask and attempt to answer an interesting question.

Stage 4: Data Acquisition and Parsing

What I have done: Submitting an automated request from R to the NYT server, pulling down the raw JSON package, and translating it into an R-readable object.

- In my Quarto code blocks, I will first load libraries (httr, jsonlite, tidyverse, janitor) in chunk 1.
- I will construct my request URL dynamically by combining the endpoint base URL with my environment variable (Sys.getenv("NYT_API_KEY")).
- I will pass that URL into jsonlite::fromJSON(..., flatten = TRUE). This converts the incoming nested JSON text file into an R list structure.

Stage 5: Tidying and Data Cleaning

- **What I have done**: Extracting the specific table of books from the broader JSON list payload, cleaning up formatting, handling missing structural pieces, and turning it into a tidy tibble ready for visualization.
- Access the inner dataframe by navigating through the list structure: response $results$books.
- Convert it to a tidy tibble (as_tibble()).
- Standardizing column headers using janitor::clean_names() (which converts everything to lower-case snake_case like weeks_on_list or rank_last_week).
- Selecting only the variables I needed for my analysis (e.g., rank, title, author, publisher, weeks_on_list).

Now lets see the code /chunks one by one

1. Load Required Libraries

The following chunk loads the essential packages needed for HTTP requests, JSON parsing, and data manipulation.

```
# Chunk 1: Setup and Libraries
library(httr)
library(jsonlite)
library(tidyverse)
library(janitor)
```

2. Securely Retrieve the API Key

This chunk verifies that my API key is correctly loaded from my environment variables without exposing sensitive credentials in plain text.

```
nyt_api_key <- Sys.getenv("NYT_API_KEY")

if (nyt_api_key == "") {
  stop("Error: NYT_API_KEY environment variable is not set. Please configure your .Renviron i
} else {
  message("API key successfully loaded from environment variables.")
}
```

3. Construct Request URL and Fetch Data

Here, I target the current Hardcover Fiction bestseller list endpoint and parse the JSON response.

```
base_url <- "https://api.nytimes.com/svc/books/v3/lists/current/hardcover-fiction.json"
request_url <- paste0(base_url, "?api-key=", nyt_api_key)

# Fetch and parse JSON response
response <- fromJSON(request_url, flatten = TRUE)

# Inspect the top-level structure of the response list
print(response$status)
```

```
[1] "OK"
```

4. Extract and Transform Nested Data into a Tidy Tibble

The raw response embeds book attributes deep inside a nested dataframe list. This chunk extracts those rows and structures them into a tidy tibble.

```
nyt_books_df <- response$results$books %>%
  as_tibble() %>%
  clean_names() %>%
  select(
    rank,
    rank_last_week,
    weeks_on_list,
    title,
    author,
    publisher,
    description,
    primary_isbn13
  ) %>%
  rename(isbn13 = primary_isbn13)

# View the first few rows of the clean dataset
head(nyt_books_df)
```

```
# A tibble: 6 x 8
   rank rank_last_week weeks_on_list title   author publisher description isbn13
  <int>          <int>         <int> <chr>   <chr>  <chr>     <chr>       <chr>
1     1              1             4 THRESH~ Rebec~ Red Tower Thirteen s~ 97816~
2     2              0             1 MISS W~ Louis~ Minotaur  The 21st b~ 97812~
3     3              0             1 A MARR~ Abby ~ Forever   A collecti~ 97815~
4     4              0             1 THE FA~ James~ Little, ~ The 36th b~ 97803~
5     5              0             1 CHAIN ~ Lee C~ Bantam    The 31st b~ 97982~
6     6              3             4 THE FR~ John ~ Doubleday When a cou~ 97803~
```

5. Data Cleaning Documentation & Summary Statistics

Data Cleaning Notes:

Nested Fields: The primary data table returned by the NYT API nested individual book metadata inside a books list frame. I extracted this sub-element directly using pipe operations.

Missing Values: rank_last_week defaults to 0 or NA for books newly entering the bestseller list. These were retained as-is to preserve list entry status.

Naming Conventions: Standardized column names to lowercase snake_case using janitor::clean_names().

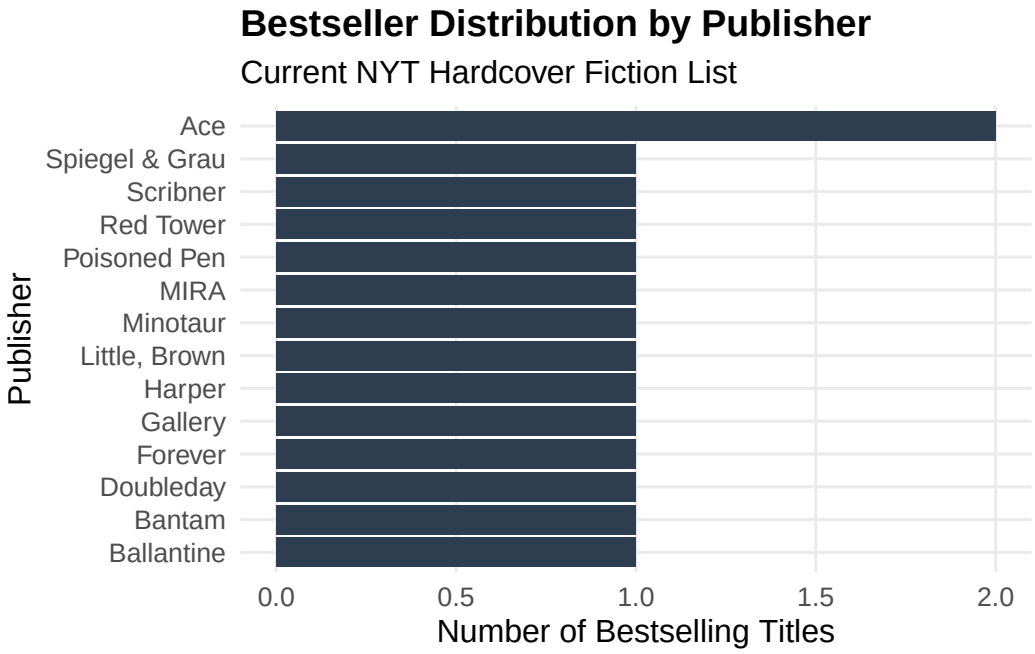
```
publisher_summary <- nyt_books_df %>%
  group_by(publisher) %>%
  summarise(
    total_books = n(),
    avg_weeks_on_list = mean(weeks_on_list, na.rm = TRUE)
  ) %>%
  arrange(desc(total_books))

print(publisher_summary)
```

```
# A tibble: 14 x 3
   publisher      total_books avg_weeks_on_list
   <chr>                <int>             <dbl>
 1 Ace                      2              26.5
 2 Ballantine               1               6
 3 Bantam                   1               1
 4 Doubleday                1               4
 5 Forever                  1               1
 6 Gallery                  1               1
 7 Harper                   1               3
 8 Little, Brown            1               1
 9 MIRA                     1               1
10 Minotaur                 1               1
11 Poisoned Pen             1               3
12 Red Tower                1               4
13 Scribner                 1               2
14 Spiegel & Grau           1              25
```

6: Data Visualization

This chunk satisfies the analytical visualization requirement by plotting publisher frequency or ranking distribution on the current bestseller list.

```
ggplot(publisher_summary, aes(x = reorder(publisher, total_books), y = total_books)) +
  geom_col(fill = "#2c3e50") +
  coord_flip() +
  labs(
    title = "Bestseller Distribution by Publisher",
```

```
    subtitle = "Current NYT Hardcover Fiction List",
    x = "Publisher",
    y = "Number of Bestselling Titles"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold"),
    panel.grid.minor = element_blank()
  )
```